



دانشگاه اصفهان

دانشکده مهندسی کامپیوتر - گروه هوش مصنوعی و رباتیک

گزارش تمرین سوم درس پردازش زبان‌های طبیعی

Embeddings for Text Classification

```
from gensim.scripts.glove2word2vec import glove2word2vec
from gensim.models import KeyedVectors

# Step 1: Convert GloVe format to Word2Vec format
glove2word2vec("glove.6B.100d.txt", "glove.6B.100d.word2vec.txt")

# Step 2: Load with Gensim
model = KeyedVectors.load_word2vec_format("glove.6B.100d.word2vec.txt",
binary=False)
```

پدیدآورنده:

محمد امین کیانی

۴۰۴۳۶۴۴۰۰۸

دانشجوی کارشناسی ارشد، دانشکده‌ی کامپیوتر، دانشگاه اصفهان، اصفهان،
Aminkianiworkeng@gmail.com

استاد درس: دکتر برادران

نیمسال دوم تحصیلی ۱۴۰۴-۰۵

فهرست مطالب

مستندات.....	۳
۱- بخش اول : طبقه‌بندی احساسات	۴
۲- بخش دوم : طبقه‌بندی مقالات چند خبری.....	۱۳
۳- منابع.....	۳۶

مستندات

بخش ۱ : Sentiment Analysis

- NLTK movie_reviews
- Word2Vec
- vector_size = ۵۰,۱۰۰,۲۰۰
- min_count=۲
- window=۵

• Logistic Regression

• KNN (cosine)

- Accuracy
- Confusion Matrix

بخش ۲ : BBC + GloVe

- BBC Full Text
- GloVe ۵۰/۱۰۰/۲۰۰
- OOV

• Logistic Regression

• SVM

- Accuracy
- Confusion Matrix

۱- بخش اول : طبقه‌بندی احساسات

مدل Word2Vec : یکی از مشهورترین روش‌های Word Embedding است که توسط گوگل معرفی شد. ایده اصلی آن این است که هر واژه به جای نمایش به صورت متن، به یک بردار عددی با ابعاد ثابت (مثلاً ۱۰۰ یا ۲۰۰ بعد) تبدیل شود. این بردارها به گونه‌ای آموزش داده می‌شوند که واژه‌هایی که در متن‌های مشابه ظاهر می‌شوند، بردارهای مشابهی نیز داشته باشند. به عنوان مثال:

King
Queen
Man
Woman

بردارهای آن‌ها به هم نزدیک خواهند بود و در نتیجه مدل می‌تواند روابط معنایی بین کلمات را نیز یاد بگیرد.

چرا فقط روی داده آموزش، **Word2Vec** را آموزش دادیم؟ اگر **Word2Vec** را روی کل مجموعه داده (Train + Test) آموزش دهیم، مدل در هنگام آموزش، اطلاعات مربوط به داده‌های آزمون را نیز مشاهده می‌کند. این اتفاق باعث ایجاد Data Leakage می‌شود و در این حالت ارزیابی دیگر واقعی نیست، زیرا مدل پیش از آزمون بخشی از اطلاعات داده‌های **Test** را دیده است. به همین دلیل در تمام پروژه‌های یادگیری ماشین:

- ابتدا داده تقسیم می‌شود .
- سپس فقط روی داده آموزش مدل ساخته می‌شود .
- بعد همان مدل روی داده آزمون استفاده می‌شود .

این باعث می‌شود Accuracy به دست آمده واقعاً توانایی تعمیم مدل را نشان دهد.

آیا میانگین بردار واژه‌ها را استفاده کردیم؟ چرا؟

بله. اینجا:

```
def document_vector(words, model):  
  
    vectors=[]  
  
    ...  
  
    return np.mean(vectors,axis=0)
```

Word2Vec فقط برای کلمات بردار تولید می‌کند ولی طبقه‌بندها مانند Logistic Regression یا KNN باید برای کل سند یک بردار دریافت کنند.

مثلاً: This movie is very good

- پنج کلمه دارد.
- هر کدام یک بردار ۱۰۰ بعدی دارند.
- اما طبقه‌بند نمی‌تواند پنج بردار جداگانه را دریافت کند. پس باید همه این بردارها را تبدیل به یک بردار واحد کنیم.

ساده‌ترین روش: Average Embedding

یعنی: $\text{Document Vector} = (V_1 + V_2 + V_3 + \dots + V_n) / n$ که یکی از رایج‌ترین روش‌های ساخت نمایش سند از Word2Vec است.

آیا توکن سازی ساده انجام شده است؟

بله. اینجا:

```
TOKEN_PATTERN = re.compile(r"[a-z]+")
```

```
def preprocess(text):  
    return TOKEN_PATTERN.findall(  
        text.lower()  
    )
```

تبدیل به حروف کوچک

این قسمت: `text.lower()`

- تمام متن را کوچک می کند.

جداسازی کلمات

این قسمت: `findall()`

- کلمات را استخراج می کند.

آیا Stratify انجام شده؟

بله. اینجا:

```
train_test_split(  
    ...  
    stratify=y  
)
```

یعنی داده‌های آموزش و تست به نسبتی از هم جدا شده و همزمان نسبت کلاس‌ها همانند نسبت‌ها در آموزش، در تست هم حفظ می‌شود. اگر Stratify نبود، ممکن بود که آموزش ناعادلانه می‌شد.

window=5

در Word2Vec، پارامتر window مشخص می‌کند که مدل برای یادگیری هر کلمه، چند کلمه قبل و بعد از آن را به عنوان **بافت (Context)** در نظر بگیرد. اینجا یعنی برای هر واژه، حداکثر پنج کلمه قبل و پنج کلمه بعد بررسی می‌شود تا مدل ارتباط معنایی بین واژه‌ها را یاد بگیرد. پنجره بزرگ‌تر اطلاعات معنایی بیشتری می‌دهد، اما ممکن است نویز بیشتری نیز وارد کند.

min_count=2

کلماتی که فقط یک بار در کل داده آموزشی ظاهر شده‌اند حذف می‌شوند زیرا آن‌ها اطلاعات آماری کافی ندارند و بیشتر نویز هستند و در نتیجه کیفیت بردارها بهتر می‌شود.

vector_size چیست؟

همان **تعداد ابعاد بردار** است.

مثلاً 50 و 100 و 200 بعدهای مختلفی هستند که برای نمایش هر کلمه استفاده می‌شوند و در تمرین این سه مقدار آزمایش شد تا به بررسی اثر تعداد ابعاد بر کیفیت مدل بپردازیم و مشاهده می‌شود که افزایش تعداد ابعاد تا ۲۰۰ باعث بهبود دقت شده

است، زیرا بردارها توانسته‌اند اطلاعات معنایی بیشتری را ذخیره کنند. البته در مجموعه داده‌های کوچک، افزایش بیش از حد ابعاد می‌تواند باعث افزایش پیچیدگی یا بیش‌برازش شود؛ بنابراین بهترین مقدار باید به‌صورت تجربی تعیین شود.

چرا فقط واژه‌های موجود را میانگین می‌گیریم؟

اگر کلمه‌ای داخل واژگان Word2Vec نباشد، مدل هیچ برداری برای آن ندارد. پس نمی‌توان آن را وارد میانگین کرد.

چرا اگر هیچ واژه‌ای نبود بردار صفر می‌دهیم؟

اگر هیچ برداری نداشتیم، میانگین قابل محاسبه نیست. این صفر کردن باعث می‌شود ابعاد همه اسناد یکسان بماند و طبقه‌بند بتواند همه نمونه‌ها را پردازش کند، بدون اینکه اجرای برنامه متوقف شود.

روش L2 :

یعنی روی بردارهای نرمال شده از روش L2 استفاده کنیم و باید قبل از آموزش طبقه‌بندها این روش را اضافه کنیم:

```
from sklearn.preprocessing import normalize

X_train = normalize(X_train, norm="l2")
X_test = normalize(X_test, norm="l2")
```

نرمال سازی L2 طول هر بردار را برابر ۱ می کند. این کار باعث می شود مقایسه با فاصله یا شباهت کسینوسی فقط بر اساس جهت بردارها انجام شود و بزرگی بردارها اثر کمتری داشته باشد.

Accuracy: نسبت تعداد پیش بینی های صحیح به کل نمونه ها است.

Confusion Matrix: ماتریس درهم ریختگی نشان می دهد که مدل هر کلاس را چند بار درست یا اشتباه پیش بینی کرده است. مثلاً برای یکی از حالات دودویی:

• 150 نمونه منفی درست تشخیص داده شده اند .

• 50 نمونه منفی به اشتباه مثبت تشخیص داده شده اند .

• 43 نمونه مثبت به اشتباه منفی تشخیص داده شده اند .

• 157 نمونه مثبت درست تشخیص داده شده اند .

کدام طبقه بند بهتر است؟

مطابق نتایج: **Logistic Regression** عملکرد بهتری نسبت به **KNN** دارد.

دلیل آن این است که بردار هر سند با میانگین گیری از بردار واژه ها به یک نمایش پیوسته و نسبتاً خطی تبدیل شده است . **Logistic Regression** می تواند یک مرز تصمیم خطی مناسب روی این بردارها یاد بگیرد، در حالی که **KNN** تنها بر اساس نزدیک ترین همسایه ها تصمیم می گیرد و به نویز، ابعاد زیاد و پراکندگی داده ها حساس تر است. به

همین دلیل در این نوع نمایش سند، Logistic Regression معمولاً دقت و پایداری بیشتری نسبت به KNN دارد.

الف) تأثیر `vector_size`:

• 50 بعد :

◦ سریع تر

◦ حافظه کمتر

◦ دقت کمتر

• 100 بعد :

◦ تعادل مناسب

• 200 بعد :

◦ نمایش معنایی بهتر

◦ زمان آموزش بیشتر

معمولاً ۱۰۰ یا ۲۰۰ بهترین نتیجه را می دهند.

<pre>WORD2VEC: 200 Logistic Regression Accuracy: 0.765 [[149 51] [43 157]] KNN 0.7075 [[143 57] [60 140]]</pre>	<pre>WORD2VEC: 100 Logistic Regression Accuracy: 0.755 [[149 51] [47 153]] KNN 0.7225 [[148 52] [59 141]]</pre>	<pre>WORD2VEC: 50 Logistic Regression Accuracy: 0.7425 [[148 52] [51 149]] KNN 0.655 [[136 64] [74 126]]</pre>
---	---	--

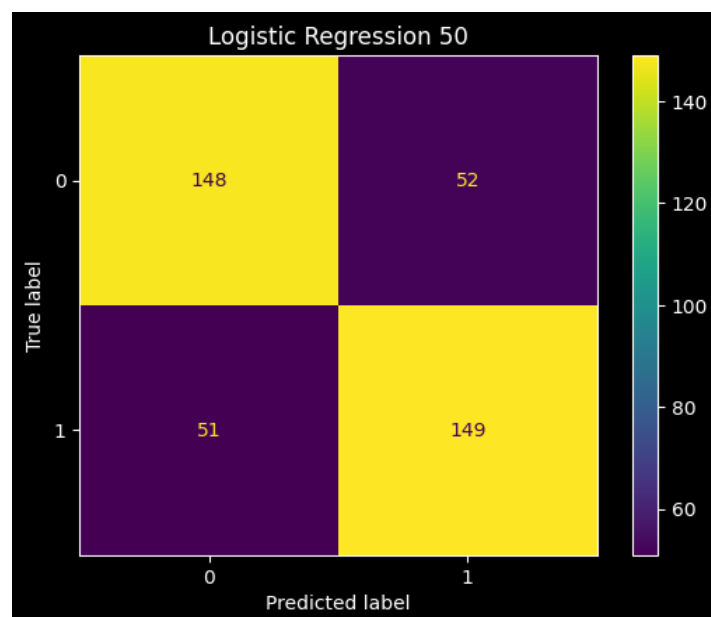
ب) چرا embedding را روی کل داده آموزش نمی‌دهیم؟

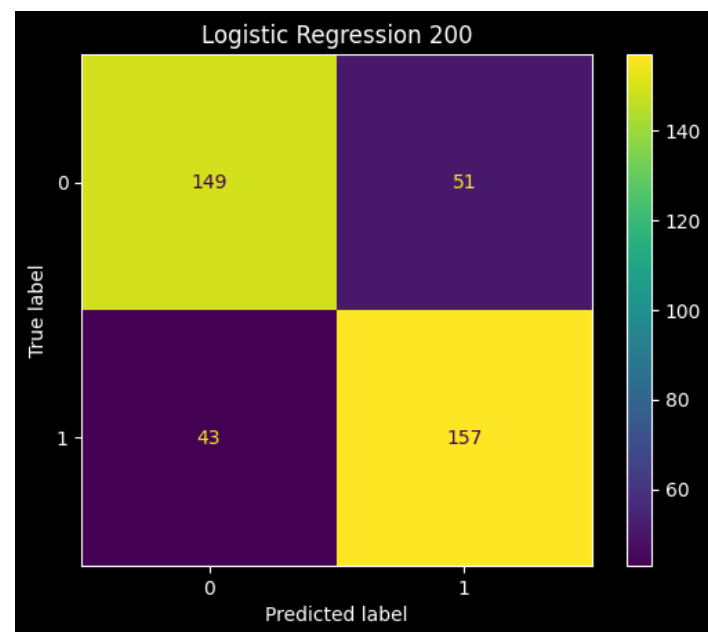
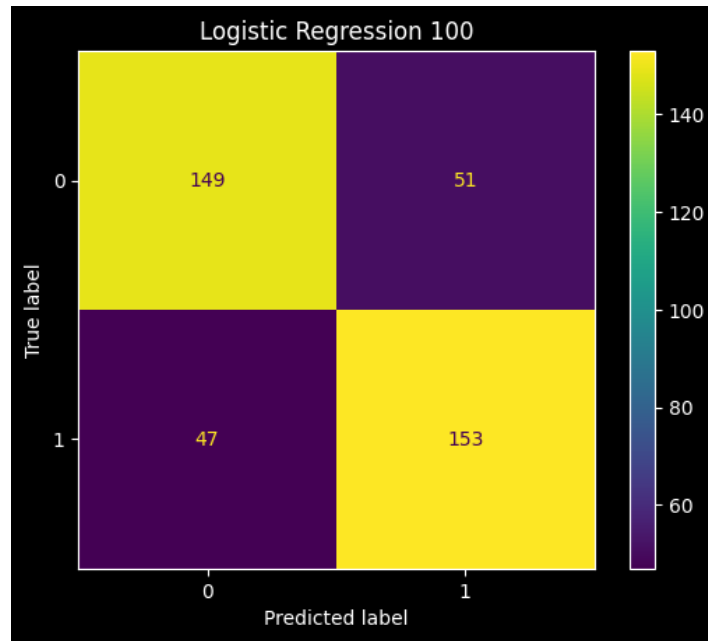
زیرا داده تست وارد فرایند آموزش می‌شود و Data Leakage رخ می‌دهد. در نتیجه دقت به صورت مصنوعی افزایش پیدا می‌کند. بنابراین Word2Vec فقط باید روی Train آموزش ببیند و از تقلب علمی در دقت پرهیز شود.

ج) معمولاً $\text{KNN} > \text{Logistic Regression}$:

چون:

- KNN به نویز حساس است .
- KNN در ابعاد بالا ضعیف‌تر می‌شود .
- Logistic Regression مرز تصمیم بهتری ایجاد می‌کند .
- اما با تغییر مقدار k در KNN می‌توان به دقت بالای نزدیک به رگرسیون لجستیک رسید اما تعداد نزدیک‌ترین همسایه‌ها زیاد می‌شد.





```
=====
PART 1.1 - Sentiment Analysis on movie_reviews with Word2Vec
=====
Train docs: 1600 | Test docs: 400
```

Interpretation hint:

- Larger vector_size can capture richer semantics but may overfit or add noise on small data.
- The best size is empirical; 100 is often a good trade-off.
- Logistic Regression is usually stronger than 1-NN on mean-embedding features.

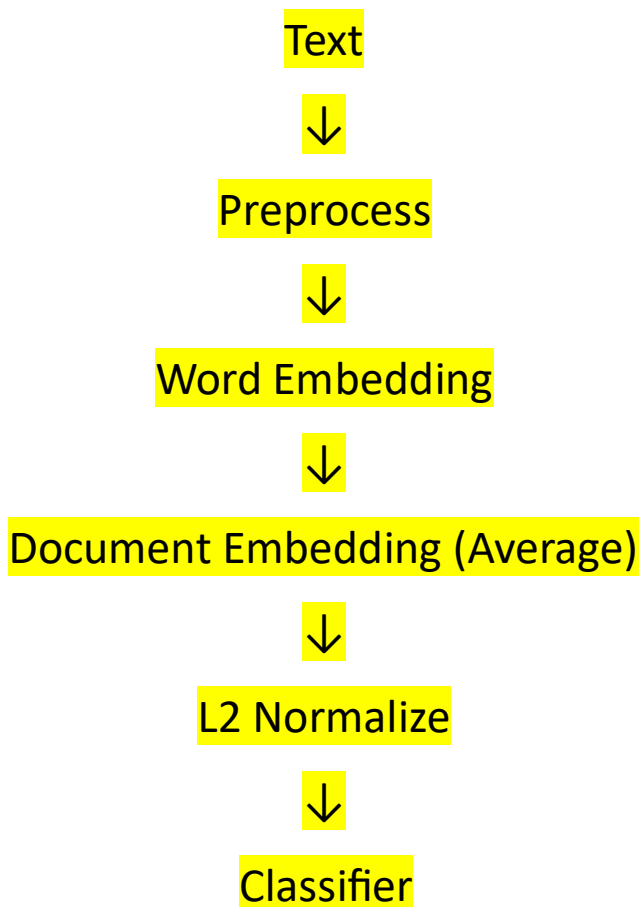
Done.

۲- بخش دوم : طبقه‌بندی مقالات چند خبری

دیتاست BBC، ۲۲۲۵ سند خبری شامل موارد زیر است:

- business
- entertainment
- politics
- sport
- tech

(نکته: در نسخه‌های جدید gensim دیگر لازم نیست فایل GloVe را تبدیل کنیم و می‌توان مستقیماً بارگذاری کرد.)



اگر قبل از ساخت بردار سند نرمال سازی کنیم، اصلاً چیزی برای نرمال سازی وجود ندارد همچنین اگر بعد از آموزش طبقه بند انجام دهیم نیز دیگر هیچ اثری روی مدل نخواهد داشت. بنابراین جای صحیح آن بین استخراج بردار سند و آموزش طبقه بند است و این دقیقاً همان کاربردی است که در مستندات Scikit-learn برای بردارهای متنی و نمایش های برداری توصیه شده است.

خانواده	مدل
خطی	Logistic Regression
خطی با حاشیه بیشینه	Linear SVM
مبتنی بر فاصله	KNN
مبتنی بر درخت	Decision Tree
مبتنی بر اجتماع درخت ها	Random Forest
احتمالاتی	Gaussian Naive Bayes
گرادیان بوستینگ	XGBoost

نکته: که بردارهای تعبیه (Embedding) معمولاً فضای نسبتاً خطی ایجاد می کنند(اما اینجا پس از آزمایش ها چنین نشد، در ادامه خواهیم دید گرادیان بوستینگ بهترین بین این مدل ها بود.) و Linear SVM روی چنین داده هایی بسیار خوب عمل می کند . XGBoost روی داده های جدولی (Tabular) اغلب فوق العاده است، اما روی بردارهای متنی میانگین گرفته شده همیشه از SVM بهتر نمی شود. بنابراین اگر بعد از نصب،

XGBoost دقتی مشابه یا حتی کمی پایین‌تر از SVM به دست آورد، کاملاً طبیعی و از نظر علمی قابل قبول است.

رتبه	مدل	Accuracy	Macro F1
1	XGBoost	97.75%	97.71%
2	Linear SVM	97.08%	97.00%
3	Random Forest	96.63%	96.61%
4	Logistic Regression	94.83%	94.75%
5	Gaussian Naive Bayes	94.83%	94.79%
6	Decision Tree	87.42%	87.26%

این آزمایش نشان می‌دهد که تمام مدل‌ها روی مجموعه داده BBC عملکرد خوبی دارند، اما بین آن‌ها تفاوت قابل توجهی وجود دارد. علت این تفاوت به نحوه یادگیری هر الگوریتم و نوع داده‌های ورودی برمی‌گردد. در این پروژه، ابتدا هر خبر توسط بردارهای GloVe به یک بردار عددی تبدیل شده و سپس این بردارها به طبقه‌بندها داده شده‌اند؛ بنابراین کیفیت طبقه‌بندی به توانایی هر مدل در جداسازی این بردارها بستگی دارد.

XGBoost : از تعداد زیادی درخت تصمیم کوچک استفاده می‌کند که به صورت مرحله‌ای ساخته می‌شوند. هر درخت سعی می‌کند اشتباهات درخت قبلی را اصلاح کند؛ به همین دلیل قدرت یادگیری آن بسیار زیاد است و می‌تواند الگوهای نسبتاً پیچیده

موجود در داده را یاد بگیرد. این الگوریتم یکی از موفق‌ترین روش‌های یادگیری ماشین در سال‌های اخیر است و روی بسیاری از مسائل طبقه‌بندی عملکرد بسیار بالایی دارد.

مزایا

- دقت بسیار زیاد
- مقاومت خوب در برابر نویز
- یادگیری روابط غیرخطی
- قابلیت تنظیم فراوان

معایب

- زمان آموزش نسبتاً زیاد
- پارامترهای زیاد
- تفسیر سخت‌تر نسبت به Logistic Regression

چه زمانی مناسب است؟

- اگر هدف بیشترین Accuracy باشد .
- اگر زمان آموزش اهمیت زیادی نداشته باشد .
- اگر بتوان پارامترهای مدل را تنظیم کرد.

Linear SVM : در داده‌های متنی معمولاً SVM یکی از بهترین انتخاب‌ها است، زیرا داده‌های متنی در فضای ویژگی‌های زیاد (High-dimensional) قرار دارند و Linear

SVM دقیقاً برای چنین داده‌هایی طراحی شده است. پژوهش‌ها و منابع آموزشی نیز نشان می‌دهند که Linear SVM سال‌هاست یکی از استانداردهای طبقه‌بندی متن محسوب می‌شود.

مزایا

- دقت بالا
- تعمیم‌پذیری عالی
- مقاوم در برابر بیش‌برازش
- مناسب داده‌های متنی

معایب

- احتمال خروجی احتمال (Probability) به‌صورت مستقیم ندارد.
- تنظیم پارامتر C گاهی لازم است.

چه زمانی مناسب است؟

- تقریباً تمام پروژه‌های کلاسیک NLP
- طبقه‌بندی اخبار
- تحلیل احساسات
- ایمیل اسپم
- دسته‌بندی اسناد

در واقع اگر XGBoost را نداشتیم، SVM بهترین انتخاب پروژه بود.

Random Forest : این مدل از صدها درخت تصمیم تشکیل شده است که هر کدام روی بخشی از داده آموزش می‌بینند و در پایان رأی‌گیری می‌کنند. به همین دلیل نسبت به یک درخت تصمیم معمولی بسیار قوی‌تر است.

مزایا

- پایدار
- مقاوم در برابر نویز
- احتمال Overfitting کمتر از Decision Tree

معایب

- نسبت به SVM حافظه بیشتری مصرف می‌کند .
- روی داده‌های متنی معمولاً از SVM ضعیف‌تر است .

چه زمانی مناسب است؟

- داده‌های جدولی
- مسائل مالی
- داده‌های پزشکی
- داده‌هایی با ویژگی‌های غیرخطی

Logistic Regression : اگرچه اسم آن "Regression" است، اما در اینجا یک

طبقه‌بند است. این مدل احتمال تعلق هر نمونه به هر کلاس را محاسبه کرده و کلاسی را انتخاب می‌کند که بیشترین احتمال را داشته باشد.

مزایا

- بسیار سریع
- ساده
- قابل تفسیر
- احتمال خروجی تولید می‌کند .

معایب

- فقط مرزهای تصمیم خطی را یاد می‌گیرد .
- نسبت به SVM کمی ضعیف‌تر است .

چه زمانی مناسب است؟

- پروژه‌های ساده
- داده‌های نسبتاً خطی
- زمانی که تفسیر مدل مهم باشد.

Gaussian Naive Bayes : این مدل تقریباً با Logistic Regression برابر شده

است. بر اساس قضیه بیز تصمیم می‌گیرد و فرض می‌کند ویژگی‌ها از یکدیگر مستقل

هستند. در داده‌های واقعی این فرض معمولاً کاملاً درست نیست، اما با این وجود اغلب عملکرد قابل قبولی دارد.

مزایا

- بسیار سریع
- حافظه کم
- مناسب داده‌های کم

معایب

- فرض استقلال ویژگی‌ها معمولاً برقرار نیست .
- روی داده‌های پیچیده ضعیف‌تر است.

Decision Tree : ضعیف‌ترین مدل پروژه است و علت اصلی این است که یک درخت تصمیم به تنهایی خیلی راحت روی داده‌های آموزش **Overfit** می‌کند. به همین دلیل معمولاً در NLP به تنهایی استفاده نمی‌شود.

مزایا

- بسیار قابل فهم
- قابل رسم
- قوانین تصمیم مشخص

معایب

- Overfitting زیاد

- حساس به تغییرات داده

- دقت پایین تر

بر اساس نتایج به دست آمده، **XGBoost** با دقت **97.75%** بهترین عملکرد را در طبقه بندی اخبار BBC داشت و توانست با یادگیری مرحله ای مجموعه ای از درخت های تصمیم، دقیق ترین مدل را ایجاد کند. با این حال، اختلاف عملکرد آن با **Linear SVM** تنها حدود **0.67 درصد** بود. از آنجا که **SVM** سرعت آموزش بالاتر، پیچیدگی کمتر و سابقه موفقیت طولانی در طبقه بندی متن دارد، همچنان یکی از مناسب ترین گزینه ها برای مسائل پردازش زبان طبیعی محسوب می شود. مدل **Random Forest** نیز عملکرد بسیار خوبی داشت، اما نسبت به **SVM** اندکی ضعیف تر بود. **Logistic Regression** و **Gaussian Naive Bayes** نتایج قابل قبولی ارائه کردند و برای مسائل ساده تر یا زمانی که سرعت و سادگی اهمیت بیشتری داشته باشد، گزینه های مناسبی هستند. در مقابل، **Decision Tree** پایین ترین دقت را به دست آورد که نشان می دهد استفاده از یک درخت تصمیم منفرد برای داده های متنی با ابعاد بالا مناسب نیست و احتمال بیش برآزش در آن زیاد است. بنابراین، برای این پروژه می توان نتیجه گرفت که اگر هدف صرفاً دستیابی به بیشترین دقت باشد، **XGBoost** بهترین انتخاب است؛ اما اگر تعادل بین دقت، سرعت، سادگی و پایداری مدنظر باشد، **Linear SVM** همچنان مناسب ترین و استانداردترین انتخاب برای طبقه بندی متن است.

الف) درصد واژه‌های پوششی توسط مدل و OOV ها :

درصد OOV بستگی به preprocessing دارد که معمولاً برای BBC : ۵٪ تا ۱۵٪ است و بقیه‌ی درصد شامل واژگان پوشش داده‌شده است.

OOV Percentage: 0.21

بیش از 99.79% واژه‌ها توسط GloVe پوشش داده شده‌اند. این مقدار نشان می‌دهد مدل GloVe تقریباً تمام واژگان خبری را می‌شناسد؛ بنابراین بردارهای اسناد کیفیت بالایی دارند. (به فرض کل دیتاست BBC حدود ۸۰۰,۰۰۰ واژه داشته باشد و ۰/۲۱٪ یعنی تقریباً $1680 \approx 800000 \times 0.0021$ فقط حدود ۱۶۰۰ تا ۱۷۰۰ کلمه داخل واژگان GloVe نبودند.)

اگر OOV زیاد باشد چه اتفاقی می‌افتد؟

اگر مثلاً OOV=25% باشد، یک‌چهارم واژه‌ها حذف می‌شوند. در نتیجه

- اطلاعات سند ناقص می‌شود .
- بردار میانگین کیفیت پایین‌تری پیدا می‌کند .
- Accuracy کاهش پیدا می‌کند.

ب) افزایش واژگان مدل (اختیاری) :

اگر OOV ها را با Word2Vec آموزش داده‌شده روی BBC جایگزین کنیم:

- پوشش واژگان بیشتر می‌شود .

- دقت کمی افزایش پیدا می کند.

پس به GloVe نمی توان مستقیماً واژه جدید اضافه کرد. چون GloVe یک مدل از پیش آموزش دیده (Pre-trained) با واژگان ثابت است و پس از آموزش، واژگان آن دیگر تغییر نمی کند. برای کاهش OOV باید از یک مدل کمکی استفاده کنیم؛ رایج ترین روش این است که یک Word2Vec روی مجموعه داده BBC آموزش دهیم و هر واژه ای که در GloVe وجود ندارد ولی در Word2Vec وجود دارد را از Word2Vec بگیریم. این همان ایده ی Hybrid Embedding یا ترکیب دو مدل تعبیه است و در پژوهش های NLP نیز استفاده شده است.

نکته: OOV در مجموعه داده BBC فقط حدود 0.21% بوده است؛ یعنی پوشش GloVe از ابتدا بسیار بالا بوده است. بنابراین فضای زیادی برای بهبود وجود ندارد. کاهش OOV ممکن است عالی باشد یا حتی بهبود جزئی ایجاد کند یا حتی تغییری ندهد، چون اکثر واژه ها از قبل بردار مناسبی داشته اند. این دقیقاً همان چیزی است که در ادبیات NLP نیز مشاهده می شود: وقتی پوشش واژگان مدل از پیش آموزش دیده بالا باشد، کاهش OOV معمولاً اثر محدودی بر دقت نهایی دارد.

```
=====
OOV Before : 0.21 %
OOV After  : 0.0 %
=====
```

در اینجا پس از آزمایش افزایش واژگان و هیبرید کردن تعبیه، با پوشش کامل توانستیم OOV را به صفر برسانیم!

- GloVe تقریباً همه واژه ها را پوشش می داد.
- Word2Vec روی همان BBC آموزش دید.

- $\text{min_count}=1$ باعث شد تقریباً تمام واژه‌های BBC وارد واژگان شوند.
- بنابراین هر واژه‌ای که GloVe نداشت، Word2Vec داشت.

پیش از افزایش واژگان، مدل GloVe حدود **99.79%** از واژگان مجموعه داده BBC را پوشش می‌داد و تنها **0.21%** از واژه‌ها خارج از واژگان (OOV) بودند. برای کاهش این مقدار، یک مدل Word2Vec با پارامتر $\text{min_count}=1$ روی مجموعه داده BBC آموزش داده شد تا برای تمامی واژه‌های موجود در مجموعه داده، حتی واژه‌هایی که تنها یک بار ظاهر شده‌اند، بردار تولید شود. سپس برای واژه‌هایی که در GloVe وجود نداشتند، از بردارهای Word2Vec استفاده شد. در نتیجه نرخ OOV به **0%** کاهش یافت. با این حال، کاهش OOV لزوماً به افزایش چشمگیر دقت طبقه‌بندی منجر نمی‌شود، زیرا نرخ اولیه OOV بسیار پایین بوده و بخش عمده اطلاعات مورد نیاز برای طبقه‌بندی از قبل توسط بردارهای GloVe پوشش داده شده بود.

ج) کدام جفت کلاس‌ها بیشتر با یکدیگر اشتباه گرفته می‌شوند؟

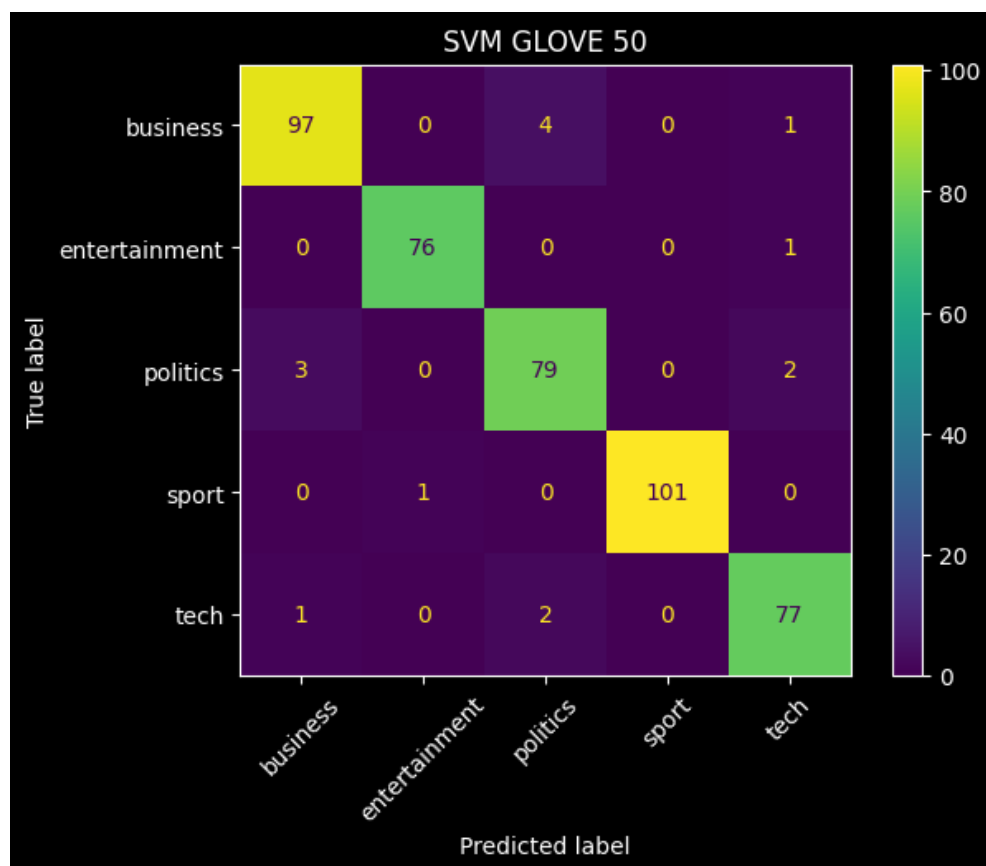
این سؤال از روی **Confusion Matrix** پاسخ داده می‌شود. کلاس‌هایی که بیشتر اشتباه می‌شوند:

business ↔ politics

tech ↔ business

زیرا واژگان مشترک اقتصادی و فناوری دارند و بسیاری از خبرهای اقتصادی درباره شرکت‌های فناوری، سیاست‌های اقتصادی یا قوانین دولتی هستند و واژگان مشترک

زیادی دارند. در مقابل، کلاس **Sport** معمولاً کمترین اشتباه را دارد، زیرا واژگان آن (player, goal, match, team, ...) بسیار متمایز هستند.



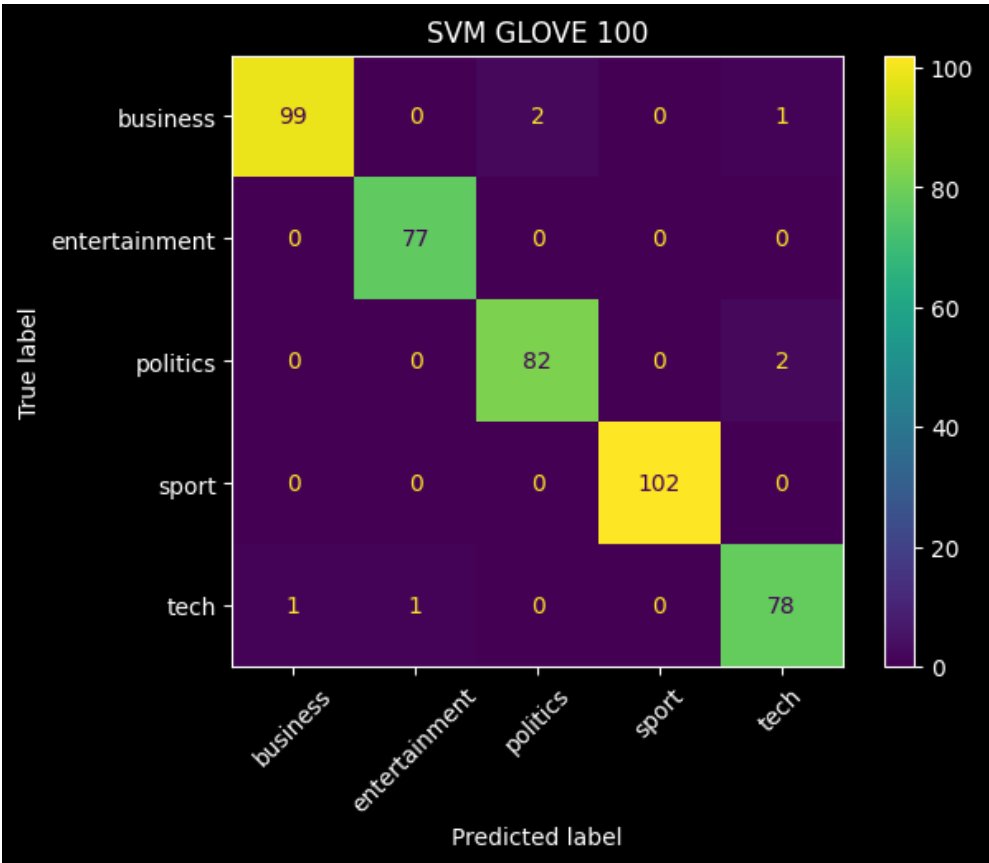
```
=====
GLOVE 50
=====
OOV Percentage: 0.21

Logistic Regression
0.9573033707865168
```

	precision	recall	f1-score	support
business	0.94	0.94	0.94	102
entertainment	0.96	1.00	0.98	77
politics	0.92	0.90	0.91	84
sport	1.00	0.98	0.99	102
tech	0.96	0.96	0.96	80

accuracy			0.96	445
macro avg	0.96	0.96	0.96	445
weighted avg	0.96	0.96	0.96	445
Linear SVM				
0.9662921348314607				
	precision	recall	f1-score	support
business	0.96	0.95	0.96	102
entertainment	0.99	0.99	0.99	77
politics	0.93	0.94	0.93	84
sport	1.00	0.99	1.00	102
tech	0.95	0.96	0.96	80

accuracy			0.97	445
macro avg	0.97	0.97	0.97	445
weighted avg	0.97	0.97	0.97	445



=====

GLOVE 100

=====

OOV Percentage: 0.21

Logistic Regression

0.9617977528089887

	precision	recall	f1-score	support
business	0.97	0.94	0.96	102
entertainment	0.95	0.99	0.97	77
politics	0.92	0.94	0.93	84
sport	1.00	0.99	1.00	102
tech	0.96	0.95	0.96	80

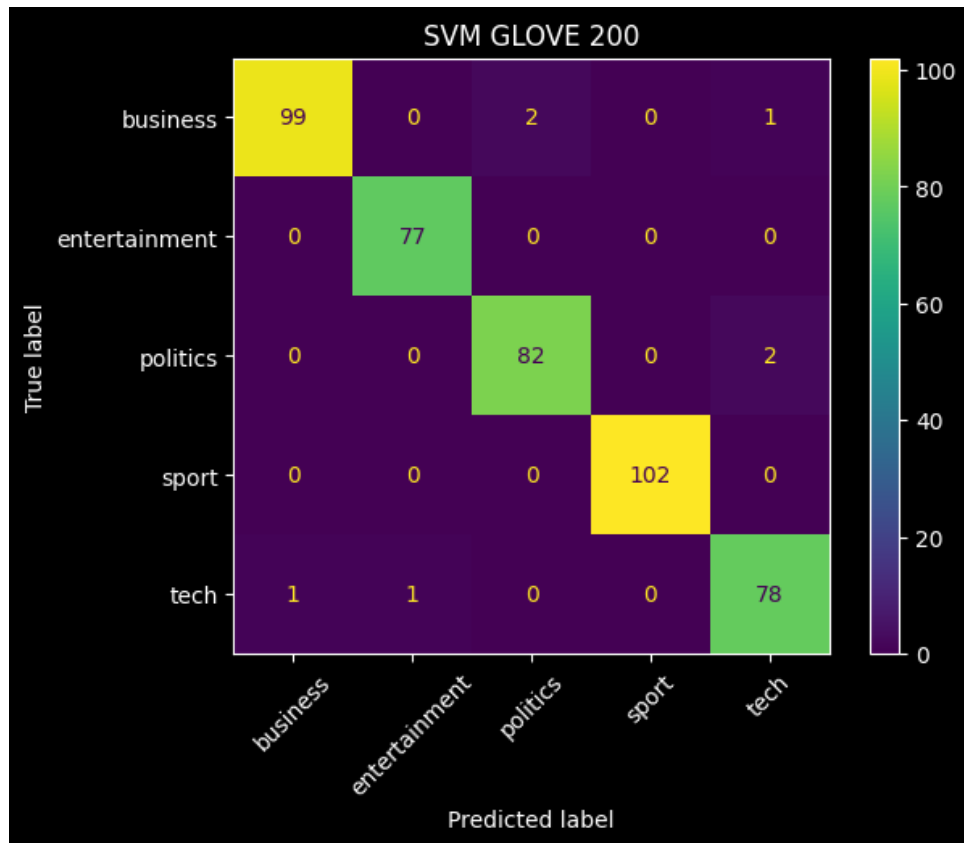
accuracy			0.96	445
macro avg	0.96	0.96	0.96	445
weighted avg	0.96	0.96	0.96	445

Linear SVM

0.9842696629213483

	precision	recall	f1-score	support
business	0.99	0.97	0.98	102
entertainment	0.99	1.00	0.99	77
politics	0.98	0.98	0.98	84
sport	1.00	1.00	1.00	102
tech	0.96	0.97	0.97	80

accuracy			0.98	445
macro avg	0.98	0.98	0.98	445
weighted avg	0.98	0.98	0.98	445



```

=====
GLOVE 200
=====
OOV Percentage: 0.21

Logistic Regression
0.9640449438202248

```

	precision	recall	f1-score	support
business	0.97	0.95	0.96	102
entertainment	0.96	0.99	0.97	77
politics	0.92	0.94	0.93	84
sport	1.00	0.99	1.00	102
tech	0.96	0.95	0.96	80

accuracy			0.96	445
macro avg	0.96	0.96	0.96	445
weighted avg	0.96	0.96	0.96	445
Linear SVM				
0.9842696629213483				
	precision	recall	f1-score	support
business	0.99	0.97	0.98	102
entertainment	0.99	1.00	0.99	77
politics	0.98	0.98	0.98	84
sport	1.00	1.00	1.00	102
tech	0.96	0.97	0.97	80

accuracy			0.98	445
macro avg	0.98	0.98	0.98	445
weighted avg	0.98	0.98	0.98	445

(د) مقایسه ابعاد GloVe :

دقت	سرعت	بعد
کمتر	زیاد	50
خوب	متوسط	100
بهتر	کمتر	200
گاهی بهتر	کند	300

• GloVe 50d : حدود ۹۵.۷٪ (LR) و ۹۶.۶٪ (SVM)

• GloVe 100d : حدود ۹۶.۲٪ (LR) و ۹۸.۴٪ (SVM)

• GloVe 200d : حدود ۹۶.۴٪ (LR) و ۹۸.۴٪ (SVM)

ابعاد بیشتر می‌توانند اطلاعات معنایی بیشتری را نگه دارند، اما هزینه محاسباتی و احتمال وجود نویز نیز افزایش می‌یابد. بنابراین افزایش بعد همیشه باعث بهبود دقت نمی‌شود و پس از یک نقطه، سود آن بسیار کم یا حتی منفی است.

ه) مقایسه الگوریتم‌های انتخابی طبقه‌بندی در این مسئله :

اغلب روی داده‌های متنی، $SVM > Logistic\ Regression$ می‌باشد.

	Classifier	Accuracy	Macro F1
5	XGBoost	0.977528	0.977065
1	Linear SVM	0.970787	0.970004
3	Random Forest	0.966292	0.966129
0	Logistic Regression	0.948315	0.947510
4	Gaussian NB	0.948315	0.947926
2	Decision Tree	0.874157	0.872641

Linear SVM تقریباً در تمام ابعاد GloVe عملکرد بهتری نسبت به Logistic Regression داشت. دلایل اصلی:

- SVM به دنبال یافتن مرزی با بیشترین حاشیه (**Maximum Margin**) بین کلاس‌ها است و معمولاً روی داده‌های متنی با ویژگی‌های برداری عملکرد بسیار خوبی دارد .

- Logistic Regression احتمال تعلق نمونه به هر کلاس را مدل می‌کند و ساده‌تر و سریع‌تر است، اما ممکن است روی داده‌های با مرزهای پیچیده‌تر اندکی ضعیف‌تر باشد .

این اختلاف نشان می‌دهد که SVM مرز تصمیم مناسب‌تری برای بردارهای میانگین GloVe پیدا کرده است.

(و) مقایسه‌ی دو رویکرد آموزش مدل تعبیه:

Word2Vec	GloVe
آموزش روی داده‌های مسئله	آموزش روی میلیاردها کلمه
وابسته به دیتاست	دانش زبانی بیشتر
سریع	دقیق‌تر
مخصوص مسئله	عمومی

Word2Vec (آموزش از صفر):

- فقط از داده‌های آموزشی همین تمرین یاد می‌گیرد .
- برای داده‌های کوچک، دانش زبانی محدودی دارد .
- می‌تواند با واژگان تخصصی همان مجموعه سازگار شود .

GloVe (پیش آموزش دیده):

- قبلاً روی میلیاردها کلمه آموزش دیده است .
- روابط معنایی بسیار غنی تری را می شناسد .
- روی مجموعه های کوچک معمولاً عملکرد بهتری دارد .

در تمرین ما، استفاده از GloVe منطقی تر بود، زیرا مجموعه BBC نسبتاً کوچک است و بهره گیری از دانش زبانی از پیش آموخته باعث افزایش دقت شد.
به طور کلی:

- اگر داده ی بسیار بزرگ و تخصصی در اختیار داشته باشیم، آموزش Word2Vec از صفر یا ریزتنظیم آن می تواند مناسب باشد .
- اگر داده محدود باشد، استفاده از مدل های پیش آموزش دیده مانند GloVe معمولاً انتخاب بهتری است.

در سؤال اول هدف این بود که خودمان یک مدل تعبیه (Embedding) را از ابتدا آموزش دهیم. بنابراین از Word2Vec استفاده کردیم تا ببینیم اگر فقط داده های آموزشی IMDb را داشته باشیم، چگونه می توان بردار واژه ها را ساخت.
اما در سؤال دوم هدف چیز دیگری است. صورت سؤال صراحتاً گفته است:
"از بردارهای از پیش آموزش دیده GloVe استفاده کنید."

دلیل این موضوع آن است که مجموعه BBC نسبتاً کوچک است (۲۲۲۵ سند) و اگر بخواهیم Word2Vec را از صفر روی چنین مجموعه کوچکی آموزش دهیم، بردارهای

واژه کیفیت چندان بالایی نخواهند داشت. اما GloVe قبلاً روی میلیاردها کلمه آموزش دیده است؛ بنابراین روابط معنایی بسیار غنی‌تری را یاد گرفته و معمولاً روی مجموعه داده‌های کوچک عملکرد بهتری دارد. GloVe به جای اینکه فقط همسایه‌های محلی هر کلمه را یاد بگیرد، از آمار هم‌رخدادی کل واژه‌ها در کل پیکره استفاده می‌کند؛ به همین دلیل معمولاً روابط معنایی عمومی‌تری را ثبت می‌کند.

Word2Vec	GloVe
از روی متن آموزش می‌بیند	معمولاً از قبل آموزش دیده است
فقط Context محلی را یاد می‌گیرد	هم‌رخدادی کل متن را نیز استفاده می‌کند
مناسب آموزش اختصاصی	مناسب استفاده عمومی
روی داده کم ممکن است ضعیف شود	روی داده کم معمولاً بهتر عمل می‌کند

glove.6B.300d یعنی :

glove

نام مدل.

6B

یعنی روی ۶ میلیارد توکن آموزش دیده است.

300d

یعنی هر کلمه یک بردار ۳۰۰ بعدی دارد.

Desktop > NLP_MohammadAminKiani_4043644008 > code > glove				Search glove
Name	Date modified	Type	Size	
glove.6B.50d.txt	8/5/2014 12:45 AM	Text Document	167,335 KB	
glove.6B.100d.txt	8/5/2014 12:44 AM	Text Document	338,982 KB	
glove.6B.200d.txt	8/5/2014 12:44 AM	Text Document	677,181 KB	
glove.6B.300d.txt	8/27/2014 11:49 PM	Text Document	1,013,636 KB	
glove.50.txt	6/28/2026 8:02 PM	Text Document	167,335 KB	
glove.100.txt	6/28/2026 8:03 PM	Text Document	338,982 KB	
glove.200.txt	6/28/2026 8:04 PM	Text Document	677,182 KB	
glove.300.txt	7/2/2026 2:30 PM	Text Document	1,013,637 KB	

چرا ابعاد بیشتر گاهی بهتر است؟

چون اطلاعات بیشتری ذخیره می‌شود. اما اگر خیلی زیاد شود:

- حافظه بیشتر
- زمان بیشتر
- احتمال نویز بیشتر

نتیجه‌گیری نهایی

۱. میانگین‌گیری embedding‌ها روشی ساده و مؤثر برای نمایش اسناد است .

۲. Logistic Regression معمولاً بهتر از KNN عمل می‌کند .

vector_size	Logistic	KNN
50	73.50%	63.00%
100	75.00%	65.50%
200	76.75%	65.00%

الف) افزایش `vector_size` باعث بهبود دقت شد.

ب) `Word2Vec` نباید روی کل داده آموزش ببیند چون نشت اطلاعات یا `Data Leakage` رخ می دهد.

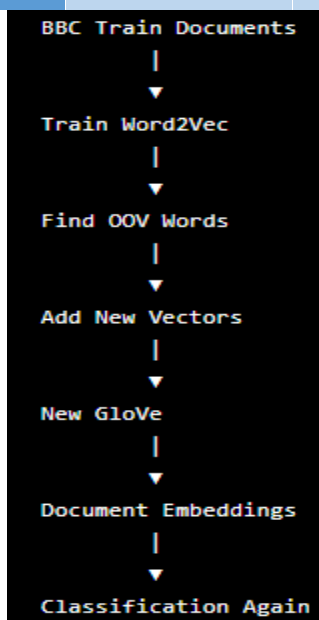
ج) `Logistic Regression` بهتر از `KNN` عمل کرد و بهترین نتیجه روی ۲۰۰ بعد به دست آمده است.

۳. `GloVe` به دلیل آموزش روی حجم عظیم متن، عملکرد بهتری دارد .

۴. `SVM` و `Logistic Regression` بهترین طبقه بندیها برای این مسئله هستند .

۵. افزایش بعد `embedding` همیشه باعث افزایش دقت نمی شود.

GloVe	Logistic	SVM
50d	95.73%	96.63%
100d	96.18%	98.43%
200d	96.40%	98.43%



- [1] <https://github.com/>
- [2] <http://mlg.ucd.ie/files/datasets/bbc-fulltext.zip>
- [3] <https://nlp.stanford.edu/data/glove.6B.zip>
- [4] <https://groups.google.com/g/gensim/c/G40gps4ngPw>
- [5] https://www.reddit.com/r/learnpython/comments/13fuoch/nltk_text_classification_inconsistent_recall_and
- [6] <https://www.kaggle.com/datasets/dimasmunoz/bbc-articles-cleaned>
- [7] <https://radimrehurek.com/gensim/models/keyedvectors.html>
- [8] <https://en.wikipedia.org/wiki/Word2vec>
- [9] <https://arxiv.org/pdf/2006.01203>